

RELATÓRIO OPERACIONAL

Mini Radar ENEM

Disciplina: Computação em Nuvem

Aula 7 — Do protótipo à produção:

operação, segurança, observabilidade e recuperação

Tecnologias: Python 3.13 · Flask 3.1 · Docker 29.8

Ambiente: Linux — Docker Engine (IaaS simulado)

Data de Execução: 24 de setembro de 2026

Sumário

1. Descrição do Serviço
2. Arquitetura e Diagrama de Exposição
3. Etapa 1 — Preparação e Inspeção do Ambiente
4. Etapa 2 — Configuração para Produção (Variáveis de Ambiente)
5. Etapa 3 — Health Check e Observabilidade
6. Etapa 4 — Segurança e Exposição
7. Etapa 5 — Registro do Incidente e Recuperação
8. Etapa 6 — Atualização para Nova Versão (v3) e Rollback
9. Reflexão Final
10. Checklist de Entrega

1. Descrição do Serviço

O Mini Radar ENEM é uma aplicação web em Python/Flask voltada para consulta de notas do ENEM, classificação de pontuações e estimativa de notas de corte. A aplicação é empacotada em containers Docker para simulação de implantação em ambiente de nuvem (IaaS/PaaS).

Atributo	Valor
Linguagem	Python 3.13
Framework	Flask 3.1
Containerização	Docker Engine 29.8.1
Porta de Serviço	5000 (HTTP)
Modo de Operação	Container Docker em IaaS Linux

2. Arquitetura e Diagrama de Exposição

A arquitetura segue o modelo de container único expondo uma aplicação Flask na porta 5000. O container roda sobre o Docker Engine instalado num host Linux, simulando um ambiente IaaS.

```
+-----+
|   Usuário Final   |
+-----+
      |
      | HTTP (Porta 5000 Exposta)
      v
+-----+
| Host Linux / Docker Engine |
| +-----+ |
| | Container: radar-enem   | |
| | (Flask Web App - v3)    | |
| | Endpoints:              | |
| | GET /                   | |
| | GET /health             | |
| | GET /aluno/<nome>        | |
| | GET /nota/<int:nota>     | |
| | POST /exibir_nota       | |
| +-----+ |
+-----+
```

3. Etapa 1 — Preparação e Inspeção do Ambiente

Comandos executados para validação do ambiente inicial:

```
$ docker --version
Docker version 29.8.1, build 4a63305

$ docker images | grep radar
radar-enem:v1    013ba8346372    176MB
radar-enem:v2    81c2229913fb    197MB

$ ls -la
app.py  Dockerfile  docker-compose.yml  requirements.txt  ...
```

Resultado da Inspeção

- Docker instalado e funcional: versão 29.8.1.
- Código-fonte presente na pasta raiz (app.py, Dockerfile, requirements.txt).
- Imagem base funcional: python:3.13-slim.
- Versões anteriores identificadas: radar-enem:v1, radar-enem:v2.

4. Etapa 2 — Configuração para Produção

A aplicação utiliza 100% de configuração externa via variáveis de ambiente, mantendo o código limpo e sem credenciais hardcoded:

Variável	Descrição	Produção	Padrão
AMBIENTE	Identificador do ambiente	producao	desenvolvimento
PORT	Porta de escuta	5000	5000
CALCULADORA_URL	Endpoint FaaS	http://host.docker.internal:8000/...	http://localhost:8000/...
APP_VERSION	Versão da app	v3.0.0	v3.0.0

Comando de Inicialização para Produção

```
$ docker run -d \  
  --name radar-enem \  
  -p 5000:5000 \  
  -e AMBIENTE=producao \  
  radar-enem:v2
```

Evidência — Container em execução

```
$ docker ps  
CONTAINER ID   IMAGE          COMMAND                  STATUS              PORTS  
a048281b1851   radar-enem:v2  "python app.py"         Up 47 seconds      0.0.0.0:5000->5000/tcp
```

5. Etapa 3 — Health Check e Observabilidade

Item	Registro / Valor Observado
Endpoint de saúde	GET /health (http://localhost:5000/health)
Resposta esperada	{"status":"healthy","versao":"v3.0.0"} — HTTP 200 OK
Porta exposta	5000 (0.0.0.0:5000->5000/tcp)
CPU observada	0.03% (em repouso)
Memória observada	25.64 MiB (0.16% do sistema)
Log relevante	* Running on all addresses (0.0.0.0); "GET /health HTTP/1.1" 200

Comandos de Observabilidade Utilizados

```
# Health check manual  
$ docker exec radar-enem python -c \  
  "import urllib.request; \  
  print(urllib.request.urlopen('http://localhost:5000/health').read().decode())"  
{"status":"healthy"}  
  
# Monitoramento de CPU e Memória  
$ docker stats radar-enem --no-stream  
CONTAINER ID   NAME          CPU %    MEM USAGE / LIMIT   MEM %    PIDS  
a048281b1851   radar-enem    0.03%   25.64MiB / 15.43GiB  0.16%    1
```

6. Etapa 4 — Segurança e Exposição

6.1 Qual porta precisa ser exposta ao usuário?

Apenas a porta da aplicação web (porta 5000 no ambiente local, ou 80/443 em nuvem por meio de um Load Balancer / Reverse Proxy com TLS).

6.2 Existe serviço que deveria ficar apenas na rede interna? Por quê?

Sim. O microserviço da Calculadora de Nota de Corte (porta 8000), bancos de dados e instâncias de cache devem ser mantidos em rede interna/privada. Isso previne acessos diretos não autorizados, desvio das regras de autenticação da API principal e ataques DoS/injeção direta.

6.3 O que aconteceria se uma credencial fosse colocada no código?

A credencial ficaria gravada no histórico do Git e nas camadas imutáveis da imagem Docker. Qualquer pessoa com acesso ao repositório ou ao Container Registry (Docker Hub / AWS ECR) poderia inspecionar a imagem (docker history ou strings) e extrair senhas e chaves secretas.

6.4 Modelo de Responsabilidade Compartilhada (IaaS)

Responsabilidade do Provedor	Responsabilidade da Equipe
Hardware físico, refrigeração, rede física, virtualização (hypervisor), segurança física do data center	Atualizações do SO, Docker Engine, segurança das imagens/containers, firewall, variáveis de ambiente, backups

7. Etapa 5 — Registro do Incidente e Recuperação

Cenário Simulado

Usuários relataram indisponibilidade total da aplicação Mini Radar ENEM. A equipe de operação iniciou o procedimento de resposta a incidentes.

Fase	Ação / Achado
Deteção	curl /health falhou com "Connection Refused". Container ausente em docker ps.
Investigação	docker ps -a e docker inspect: estado Exited (137), término 11:58:12Z.
Causa Raiz	Interrupção forçada do processo python app.py (docker stop / SIGKILL). Exit code 137.
Recuperação	Executado docker start radar-enem para restabelecer o container.
Validação	Container retornou ao status "Up". Health check respondeu HTTP 200 OK.

Evidência — Investigação e Recuperação

```
$ docker ps -a --filter "name=radar-enem"
CONTAINER ID   IMAGE          STATUS              NAMES
a048281b1851   radar-enem:v2 Exited (137) 31 seconds ago  radar-enem

$ docker inspect --format='ExitCode: {{.State.ExitCode}}' radar-enem
ExitCode: 137

$ docker start radar-enem
radar-enem

$ docker exec radar-enem python -c "...
{"status":"healthy"}
```

8. Etapa 6 — Atualização para Nova Versão (v3)

8.1 Alteração no Código-fonte (app.py)

Adicionada a chave "versao": "v3.0.0" no retorno JSON dos endpoints / e /health, permitindo identificar a versão em execução por meio da própria API.

```
@app.route("/health")
def health():
    return jsonify({
        "status": "healthy",
        "versao": os.getenv("APP_VERSION", "v3.0.0")
    })
```

8.2 Build e Deploy da Imagem v3

```
$ DOCKER_BUILDKIT=0 docker build -t radar-enem:v3 .
Successfully built 7d06b0d928b1
Successfully tagged radar-enem:v3

$ docker stop radar-enem && docker rm radar-enem
$ docker run -d --name radar-enem -p 5000:5000 -e AMBIENTE=producao radar-enem:v3
```

8.3 Validação da Nova Versão

```
$ docker exec radar-enem python -c "...
GET /      : {"disciplina":"Computacao em Nuvem","projeto":"Radar ENEM",
              "status":"online","versao":"v3.0.0"}
GET /health: {"status":"healthy","versao":"v3.0.0"}
```

8.4 Procedimento de Rollback

Se a v3 apresentar problemas em produção, rollback imediato para v2:

```
$ docker stop radar-enem && docker rm radar-enem
$ docker run -d --name radar-enem -p 5000:5000 -e AMBIENTE=producao radar-enem:v2
```

9. Reflexão Final

1. Qual foi a principal diferença entre simplesmente executar um container e operar uma aplicação?

Executar um container é apenas disparar um comando (docker run). Operar uma aplicação exige planejamento de ciclo de vida completo: configuração via ambiente, monitoramento contínuo de consumo (CPU/RAM), observabilidade de logs, tratamento de falhas (diagnóstico e recuperação), controle de versões e gestão de segurança.

2. Como o health check ajuda a detectar problemas?

O health check fornece uma sonda automatizada e constante do estado da aplicação. Em vez de esperar que usuários reclamem, orquestradores (Docker, Kubernetes, AWS ECS) consultam o endpoint /health periodicamente e podem reiniciar containers doentes ou redirecionar tráfego automaticamente.

3. Por que os logs são importantes durante um incidente?

Os logs registram o histórico de execução, requisições recebidas, exceções e stack traces. Sem logs, a investigação torna-se adivinhação; com logs, identifica-se exatamente a linha do código ou o evento externo que provocou a falha.

4. O que deve ser observado antes de aumentar ou reduzir recursos?

Deve-se analisar o consumo real histórico de CPU, memória, I/O de disco e banda de rede via ferramentas de observabilidade (docker stats / Prometheus). Aumentar sem gargalos comprovados gera custos desnecessários; reduzir sem margem pode levar a crashes por OOM.

5. Qual configuração foi retirada do código e transformada em configuração externa?

Retiramos a especificação do ambiente (AMBIENTE=producao), a versão da aplicação (APP_VERSION=v3.0.0), a porta (PORT=5000) e a URL do microserviço (CALCULADORA_URL).

6. Diferença entre esse processo em uma VM IaaS e em uma plataforma PaaS?

Em IaaS (ex: AWS EC2), a equipe gerencia SO, Docker Engine, firewall e scripts de reinicialização. Em PaaS (ex: Heroku, Cloud Run), a infraestrutura é abstraída; fornecemos apenas a imagem/código e o PaaS cuida de deploy, SSL, escalabilidade e auto-recuperação.

7. O que mudaria com várias instâncias?

Seria necessário um Load Balancer na frente das instâncias, centralização de logs (Elasticsearch/Loki/CloudWatch) e garantia de que a aplicação seja stateless (sem estado na memória local).

8. Quais responsabilidades continuariam sendo da equipe em um serviço gerenciado?

Código-fonte, qualidade da aplicação, regras de negócio, modelagem de banco de dados, variáveis de ambiente seguras, arquitetura da solução e políticas de controle de acesso.

10. Checklist de Entrega

- ✓ Aplicação funcionando e testada
- ✓ Configurações externas por variáveis de ambiente documentadas
- ✓ Endpoint de Health check validado (/health)
- ✓ Logs consultados e analisados
- ✓ CPU e memória observados via docker stats
- ✓ Análise de portas e segurança realizada
- ✓ Incidente simulado e causa identificada (ExitCode: 137)
- ✓ Serviço recuperado com sucesso
- ✓ Nova versão v3 construída e implantada
- ✓ Procedimento de Rollback documentado
- ✓ RUNBOOK / Documentação operacional entregue
- ✓ Diagrama de arquitetura documentado

Documento gerado automaticamente em 24/09/2026.